Virtual Daycare Platform

Project Documentation

Project Title: Virtual Daycare Platform — Little Learners Virtual Daycare

Technology Stack: Python, Flask, SQLAlchemy, Flask-SocketIO, SQLite

Document Version: 1.0

Prepared For: Academic Submission

Date: October 2026

This document presents the complete technical and functional documentation of the Virtual Daycare Platform, a web-based system designed to manage real-time communication between daycare administrators, caregivers, and parents.

Table of Contents

Chapter	Title	Page
1	**Introduction**	6
1.1	Project Overview	6
1.2	Problem Statement	7
1.3	Objectives	8
1.4	Scope of the Project	9
1.5	Organization of the Report	10
2	**Literature Review**	11
2.1	Existing Daycare Management Systems	11
2.2	Real-Time Web Communication Technologies	12

Chapter	Title	Page
2.3	Role-Based Access Control in Web Applications	13
2.4	QR Code Integration in Education Systems	13
2.5	Summary of Literature Review	14
3	**System Requirements**	15
3.1	Functional Requirements	15
3.2	Non-Functional Requirements	17
3.3	Hardware Requirements	18
3.4	Software Requirements	18
3.5	User Roles and Permissions Matrix	19
4	**System Design**	20
4.1	System Architecture Overview	20
4.2	Application Layer Diagram	21
4.3	Database Design	22
4.4	Entity Relationship Diagram	25
4.5	Use Case Diagram	26
4.6	Class Diagram	28
4.7	Sequence Diagrams	29
4.8	Data Flow Diagram	32
5	**Technology Stack**	33
5.1	Backend Framework — Flask	33
5.2	Database — SQLAlchemy with SQLite	34
5.3	Real-Time Communication — Flask-SocketIO	35
5.4	Authentication — Flask-Login	35
5.5	QR Code Generation — qrcode and Pillow	36
5.6	Frontend Technologies	36
5.7	Deployment — Gunicorn and Procfile	37
6	**Module Description**	38
6.1	Authentication Module	38
6.2	Administrator Module	40
6.3	Caregiver Module	42
6.4	Parent Module	45

Chapter	Title	Page
6.5	Messaging Module	47
6.6	Learning Resources Module	49
6.7	Notifications Module	50
6.8	REST API Module	51
7	**Database Schema and Models**	53
7.1	User Model	53
7.2	Child Model	54
7.3	ActivityLog Model	55
7.4	Attendance Model	56
7.5	Message Model	57
7.6	Notification Model	57
7.7	Photo Model	58
7.8	LearningResource Model	58
7.9	Complaint Model	59
8	**Implementation**	60
8.1	Project Structure	60
8.2	Application Factory Pattern	61
8.3	Configuration Management	62
8.4	Blueprint Architecture	63
8.5	Security Implementation	64
8.6	File Upload Handling	65
8.7	QR Code Attendance System	65
8.8	Real-Time Notification System	66
9	**Testing**	68
9.1	Testing Strategy	68
9.2	Test Case Descriptions	69
9.3	Test Results Summary	71
10	**Deployment**	72
10.1	Local Development Setup	72
10.2	Environment Configuration	73
10.3	Database Initialization	73
10.4	Production Deployment	74
11	**Conclusion and Future Work**	75
11.1	Conclusion	75

Chapter	Title	Page
11.2	Limitations	76
11.3	Future Enhancements	76
References		78
Appendix		79

Chapter 1: Introduction

1.1 Project Overview

The Virtual Daycare Platform is a full-stack web application built to modernize the operational and communicative aspects of daycare centre management. The system is named "Little Learners Virtual Daycare" and serves as a centralized hub where three categories of users — administrators, caregivers, and parents — can interact in real time through a shared digital environment.

In a traditional daycare setting, information about a child's daily activities, meals, naps, and health conditions is communicated informally through paper logs, verbal handoffs at pickup time, or at best, telephone calls. This creates significant gaps in transparency, delays in receiving critical health alerts, and an overall lack of documented accountability. Parents who work full-time particularly struggle with not knowing what is happening with their child during the day.

The Virtual Daycare Platform was conceived to address this problem through technology. By bringing together the director, teaching staff, and families onto a single platform, the system allows caregivers to log every meaningful activity as it happens — from meals and naps to developmental milestones and diaper changes. Parents receive notifications in real time, can view a chronological activity timeline for their child, browse a photo gallery, and track attendance records with check-in and check-out timestamps.

The administrator, referred to as the Director in the interface, maintains oversight of the entire operation — enrolling children, approving caregiver accounts, responding to parent inquiries, managing the learning resource library, and monitoring the dashboard for operational metrics.

The system is built on top of the Flask web framework using Python 3.11, employs SQLAlchemy for database operations, uses Flask-SocketIO for real-time WebSocket-based communication, and stores data in a SQLite database by default, with the architecture designed for easy migration to PostgreSQL or MySQL in production.

The application uses the Application Factory pattern in Flask, organizes routes using Blueprints, enforces role-based access control on every protected route, and implements in-app notifications that are delivered both persistently to the database and in real time via WebSocket events. Emergency alerts sent by caregivers trigger both an in-app notification and a simulated SMS and email dispatch, with hooks ready to be connected to Twilio and SendGrid in a production environment.

1.2 Problem Statement

Daycare centres collect and generate a significant amount of operational data every day. For each child enrolled, there are records to maintain covering attendance, food intake, rest periods, mood observations, health notes, photo documentation, and communications with parents. In most centres, these records are kept in paper form, passed verbally at the door, or sent as informal messages through personal mobile phones.

The core problems this creates are as follows.

Lack of real-time visibility: Parents have no way to check on their child's status during the day without calling the centre and interrupting staff. This is particularly stressful when a child is unwell or when a parent is anxious about a new child's adjustment period.

Communication gaps: Important information such as a fever, a change in mood, or a dietary refusal can be lost between staff shifts or simply not communicated until pickup time.

No structured documentation: Without a digital log, there is no historical record of a child's development, attendance patterns, or health observations that can be shared with paediatricians or used by the centre for internal review.

Access control concerns: In a traditional phone or group chat setup, information about one child may inadvertently be shared with the wrong parent, raising privacy and data protection concerns.

Operational inefficiency: Attendance marking, caregiver assignment, complaint handling, and learning resource management all happen in disconnected channels, reducing the efficiency of the director's time.

The Virtual Daycare Platform directly addresses each of these pain points through a purpose-built, role-isolated, real-time digital system.

1.3 Objectives

The primary objectives of the Virtual Daycare Platform are as follows.

Objective 1 — Real-Time Parent Engagement: Allow parents to monitor their child's daily activities, including meals, naps, mood, diaper changes, and developmental milestones, in real time as caregivers log them. Parents should receive instant in-app notifications whenever a new activity is logged.

Objective 2 — Structured Attendance Management: Provide caregivers with a simple mechanism to mark daily attendance for each child, including precise check-in and check-out timestamps. Support QR code-based scanning as a fast alternative to manual marking. Present parents with a monthly attendance summary with percentages.

Objective 3 — Secure Messaging and Emergency Alerts: Enable a private messaging channel between the parent and caregiver for each child. Allow caregivers to issue high-priority emergency health alerts that immediately notify the parent through in-app notification, simulated SMS, and simulated email.

Objective 4 — Role-Based Access Control: Ensure that no user can access data belonging to a child that is not theirs. Parents should see only their own children's data. Caregivers should see only the children

assigned to them. Administrators have full visibility across all data.

Objective 5 — Learning Resource Library: Provide an organized library of age-appropriate learning content including nursery rhymes, stories, activity guides, and games, categorized by content type and age group, accessible to all authenticated users.

Objective 6 — Administrative Control Centre: Give the Director a comprehensive dashboard with operational metrics, caregiver approval workflows, child enrolment management, complaint handling, and learning resource administration.

Objective 7 — Media Management: Allow caregivers to upload photographs of children during activities, organized by category, which parents can browse in a private gallery view.

1.4 Scope of the Project

The project scope covers the following areas.

The platform is a web-based application accessible through any modern browser on desktop or mobile devices. It is not a native mobile application, though the responsive design makes it functional on smaller screens.

The system supports three user roles: Administrator (Director), Caregiver (Caretaker), and Parent. Each role has a distinct set of permissions and a dedicated dashboard interface.

The messaging system is child-centric, meaning each message thread is tied to a specific child and connects that child's parent with their assigned caregiver. There is no general staff-wide chat or broadcast messaging in the current version.

The platform handles photo uploads for children's activity documentation but does not support live video streaming or camera feeds.

The attendance system supports manual marking through the interface and QR code scanning via a web-based scanner. It does not currently integrate with hardware badge scanners or biometric devices.

The notification system sends real-time in-app notifications via WebSocket events and simulates SMS and email dispatch through log statements and print output, with documented hooks for Twilio and SendGrid integration.

The database used in development is SQLite. The architecture supports migration to PostgreSQL or MySQL by changing a single environment variable without any code changes.

The learning resource library is managed entirely by the administrator. Parents and caregivers can browse and read resources but cannot create or edit them.

Out of scope for this version: native mobile applications, video calling, payment processing, multi-daycare network management, and government compliance report generation.

1.5 Organization of the Report

This document is organized into eleven chapters. Chapter 2 reviews existing literature and comparable systems. Chapter 3 defines the functional and non-functional requirements. Chapter 4 covers the complete system design including architecture, database design, and UML diagrams. Chapter 5 describes the technology stack in detail. Chapter 6 provides a module-by-module description of the application's features and logic. Chapter 7 documents each database model. Chapter 8 describes the implementation approach. Chapter 9 covers the testing strategy and results. Chapter 10 describes the deployment setup. Chapter 11 presents conclusions and future enhancement possibilities.

Chapter 2: Literature Review

2.1 Existing Daycare Management Systems

Several commercial daycare management solutions exist in the market today, each addressing portions of the problem that the Virtual Daycare Platform addresses holistically.

Brightwheel is a widely adopted platform used by daycare centres across North America. It offers activity logging, attendance tracking, messaging, and billing management. Brightwheel is a SaaS product with a subscription model, making it inaccessible to smaller, budget-constrained centres. Its billing and payment processing features are central to its value proposition, which is a domain the Virtual Daycare Platform intentionally excludes to remain focused on the care and communication workflow.

Tadpoles is another commercial tool that allows caregivers to post daily reports and photos. It is popular in infant and toddler rooms but lacks the QR code attendance and emergency alert features present in the Virtual Daycare Platform.

HiMama provides parent communication tools with a focus on developmental milestone documentation. It offers some real-time features but relies primarily on email digest notifications rather than WebSocket-based instant alerts.

Procare Solutions is an enterprise-grade platform used by large daycare chains. It integrates point-of-sale billing, payroll, and government compliance reporting. Its complexity and cost put it out of reach for individual centre operators.

The gap that the Virtual Daycare Platform fills is a fully customizable system that can be deployed independently by a single daycare centre without recurring subscription costs, with real-time communication built in from the ground up using WebSocket technology.

2.2 Real-Time Web Communication Technologies

Real-time communication in web applications has been approached through several different technologies over the years.

HTTP Polling is the simplest approach, where the client sends an HTTP request every few seconds to check for new data. This is resource-inefficient and introduces latency equal to the polling interval.

Long Polling is an improvement where the server holds the connection open until new data is available, then responds. This reduces unnecessary requests but still has overhead from repeated connection establishment.

Server-Sent Events (SSE) is a one-directional channel where the server can push updates to the client over a persistent HTTP connection. It is well-suited for notification feeds but does not allow the client to send data back over the same connection.

WebSockets provide a full-duplex communication channel over a single persistent TCP connection. Once established, both the server and client can send messages at any time without the overhead of HTTP headers on each exchange. This is the technology that Flask-SocketIO wraps and provides to the Virtual Daycare Platform.

The choice of Flask-SocketIO was motivated by its tight integration with the Flask ecosystem, its ability to fall back to long polling when WebSocket connections are unavailable, and its support for room-based event broadcasting, which maps naturally to the child-centric notification model of the platform.

2.3 Role-Based Access Control in Web Applications

Role-Based Access Control (RBAC) is a widely established security pattern where user permissions are determined by the role assigned to the user's account rather than by individual permission assignments. This simplifies management because adding a new user with a given role automatically grants them the appropriate set of permissions.

Flask-Login, the library used in this project, handles session management and provides the `current_user` proxy that makes the authenticated user available in every request context. Role enforcement is implemented through a custom `role_required` decorator defined in `routes/utils.py` that checks the current user's role before allowing access to any protected route.

The application defines three roles: `admin`, `caregiver` (also recognized as `caretaker` for compatibility), and `parent`. Access control is enforced at two layers: the route level through the decorator, and the data level through the `get_authorized_child` utility function, which verifies not just the user's role but their specific relationship to the requested child record.

2.4 QR Code Integration in Education Systems

QR codes have found increasing adoption in educational and institutional settings as a means of fast, contactless data capture. In library management systems, QR codes on student cards allow instant book checkout. In school attendance systems, students scan a room-specific QR code to mark their presence.

The daycare context offers a natural application: each enrolled child has a unique QR code generated at registration time. The QR code encodes the string `DAYCARE_CHECKIN:CHILD_ID=<id>`, which the server parses to determine which child to process. The caregiver scans the QR code displayed on the child's parent card or profile page, and the server automatically determines whether to perform a check-in or check-out based on the current attendance state for that day.

The `qrcode` Python library generates the QR image, and `Pillow` handles image file saving. The generated PNG files are stored under `static/uploads/qrcodes/` and are served as static files.

2.5 Summary of Literature Review

The literature review confirms that while commercial daycare management solutions exist, they tend to be either too expensive for small operators, too narrowly focused on specific features like billing, or lacking real-time communication capabilities. The Virtual Daycare Platform's design is informed by the strengths of existing systems while adding key differentiators: WebSocket-based real-time updates, QR code attendance, emergency health alerts with simulated external dispatch, strict child-level access isolation, and a fully customizable architecture.

Chapter 3: System Requirements

3.1 Functional Requirements

The following functional requirements were identified through analysis of the problem domain and the expected workflows of the three user roles.

Authentication and Account Management

FR-01: The system shall allow users to register an account by providing their full name, email address, password, and role selection (Parent or Caregiver). Admin accounts are not self-registered.

FR-02: The system shall validate the email address for uniqueness before creating a new account.

FR-03: The system shall hash all passwords using Werkzeug's generate_password_hash function. Passwords shall never be stored in plain text.

FR-04: Caregiver accounts shall require administrator approval before they can access any child records or caregiver-specific features. The caregiver shall see a pending approval page upon login until approved.

FR-05: The system shall support password reset through a time-limited token-based flow. The token expires one hour after generation.

FR-06: The system shall allow authenticated users to log out, clearing their session.

Administrator (Director) Features

FR-07: The administrator shall have access to a dashboard displaying total enrolled children, active caregivers, pending caregiver approvals, total parents, open complaints, and today's attendance percentage.

FR-08: The administrator shall be able to view, approve, or revoke approval for caregiver accounts.

FR-09: The administrator shall be able to enrol new children by entering the child's name, date of birth, gender, allergies, special notes, and assigning a parent and caregiver.

FR-10: The administrator shall be able to edit existing child profiles including reassigning the parent or caregiver.

FR-11: The administrator shall be able to view all parent complaints and inquiries, respond to them with a text response, and update the status to open, in-progress, or resolved.

FR-12: The administrator shall be able to create, edit, and delete learning resources, specifying the title, content type, age group, category, description, content body, and icon.

Caregiver Features

FR-13: Caregivers shall see a dashboard listing all children assigned to them.

FR-14: Caregivers shall be able to view a detailed profile page for any of their assigned children, including today's activity logs, recent photos, and current attendance status.

FR-15: Caregivers shall be able to log activities for their assigned children. Activity types include: meal, nap, mood, diaper change, note, and developmental milestone. Each activity type has specific input fields appropriate to that type.

FR-16: Caregivers shall be able to upload photographs for an assigned child's activity documentation. Supported formats include PNG, JPG, JPEG, GIF, and WEBP. Maximum file size is 5 MB.

FR-17: Caregivers shall be able to mark attendance for assigned children including check-in time, check-out time, manual status update, and notes.

FR-18: Caregivers shall have access to a daily attendance roster view for all their assigned children with quick check-in and check-out actions.

FR-19: Caregivers shall be able to dispatch an emergency alert for an assigned child by selecting a severity level (info, warning, or critical), entering a title, and entering a description. The alert shall create a high-priority message in the child's chat thread and dispatch a simulated SMS and email to the parent.

FR-20: The QR scanner interface shall allow caregivers to scan a child's QR code to automatically process check-in or check-out based on that day's existing attendance state.

Parent Features

FR-21: Parents shall see a dashboard showing all their enrolled children with today's activity summary, attendance status, and any active emergency alerts.

FR-22: Parents shall be able to view a day-by-day activity timeline for each of their children, navigating between dates using a date picker.

FR-23: Parents shall be able to browse a photo gallery for each of their children, filterable by category such as Art, Outdoor, Meal, Milestone, Nap, and Activity.

FR-24: Parents shall be able to view monthly attendance statistics for each of their children, including total possible days, days present, days absent, and attendance percentage.

FR-25: Parents shall be able to view their child's profile including assigned caregiver, allergies, notes, and QR code.

FR-26: Parents shall be able to submit inquiries and feedback to the daycare director through a complaint form.

Messaging

FR-27: The messaging system shall maintain a private chat thread per child between that child's parent and assigned caregiver.

FR-28: Messages shall be delivered in real time via WebSocket events without requiring a page refresh.

FR-29: Unread message counts shall be displayed on the messages navigation item.

FR-30: Messages marked as alerts shall be visually distinguished from regular messages in the chat thread.

Notifications

FR-31: The system shall generate in-app notifications for relevant events including: new activity logged, photo uploaded, attendance check-in, attendance check-out, emergency alert dispatched, caregiver account approved, child profile enrolled, new complaint submitted, and complaint responded to.

FR-32: Notifications shall be delivered in real time via WebSocket events. Unread notification count shall be visible in the navigation bar.

FR-33: Urgent notifications shall additionally trigger simulated SMS and email dispatch.

3.2 Non-Functional Requirements

NFR-01 — Security: All routes accessing child data shall enforce authentication and role-based access control. A parent shall not be able to access any data belonging to a child that is not enrolled under their account. This isolation shall be enforced at the data query level, not only at the route level.

NFR-02 — Performance: The application shall respond to standard page requests within two seconds under normal load. Real-time notification delivery via WebSocket shall occur within one second of the triggering event.

NFR-03 — Usability: The user interface shall be responsive and functional on modern desktop browsers and mobile browsers without requiring any browser plugin or native application installation.

NFR-04 — Reliability: The application shall handle database errors gracefully with session rollback on internal errors. Custom error pages shall be displayed for 403, 404, and 500 HTTP error codes.

NFR-05 — Maintainability: The codebase shall use the Flask Blueprint architecture to keep each functional area isolated in its own module. Database models shall be defined in separate files within the models package.

NFR-06 — Scalability: While the default database is SQLite, the database URI configuration shall be externalized through an environment variable, allowing replacement with PostgreSQL or MySQL without

code changes.

NFR-07 — Extensibility: External notification hooks for SMS and email shall be isolated in the `notifications.py` module with documented swap points for Twilio and SendGrid integration.

NFR-08 — Portability: The application shall run on any operating system supporting Python 3.11 or higher and shall be deployable using Gunicorn on Linux-based cloud platforms.

3.3 Hardware Requirements

Development Environment

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent, 2.0 GHz
RAM	4 GB
Storage	2 GB available disk space
Network	Internet access for package installation
Display	1280 x 720 resolution or higher

Production Server

Component	Minimum Requirement
Processor	2 virtual CPU cores
RAM	1 GB (2 GB recommended)
Storage	10 GB SSD
Network	100 Mbps bandwidth
Operating System	Ubuntu 20.04 LTS or Debian 11

3.4 Software Requirements

Component	Version	Purpose
Python	3.11 or higher	Runtime language
Flask	3.0.x	Web framework
Flask-SQLAlchemy	3.1.x	ORM and database abstraction
Flask-Login	0.6.x	Session management and authentication
Flask-SocketIO	5.3.x	WebSocket and real-time event handling

Component	Version	Purpose
simple-websocket	1.0.x	WebSocket transport layer
Werkzeug	3.0.x	Security utilities and WSGI layer
python-dotenv	1.0.x	Environment variable loading
qrcode	8.0+	QR code image generation
Pillow	10.0.x	Image processing and file saving
Gunicorn	21.2.x	Production WSGI server
SQLite	Built-in	Development database
Git	2.x	Version control

3.5 User Roles and Permissions Matrix

Feature	Admin	Caregiver	Parent
Admin Dashboard	Yes	No	No
Enrol / Edit Child	Yes	No	No
Approve Caregivers	Yes	No	No
Manage Learning Resources	Yes	No	No
Respond to Complaints	Yes	No	No
View Caregiver Dashboard	Yes (all)	Yes (assigned)	No
Log Activities	Yes	Yes (assigned)	No
Upload Photos	Yes	Yes (assigned)	No
Mark Attendance	Yes	Yes (assigned)	No
QR Code Scanning	Yes	Yes (assigned)	No
Emergency Alert	Yes	Yes (assigned)	No
View Parent Dashboard	Yes (all)	No	Yes (own)
View Activity Timeline	Yes (all)	No	Yes (own)
Browse Photo Gallery	Yes (all)	No	Yes (own)
View Attendance Records	Yes (all)	No	Yes (own)
Submit Complaints	No	No	Yes
View Own Complaints	No	No	Yes
Send Messages	Yes	Yes (assigned)	Yes (own)
View Notifications	Yes	Yes	Yes
Browse Learning Resources	Yes	Yes	Yes

Chapter 4: System Design

4.1 System Architecture Overview

The Virtual Daycare Platform follows a three-tier architecture consisting of the presentation tier, the application logic tier, and the data tier.

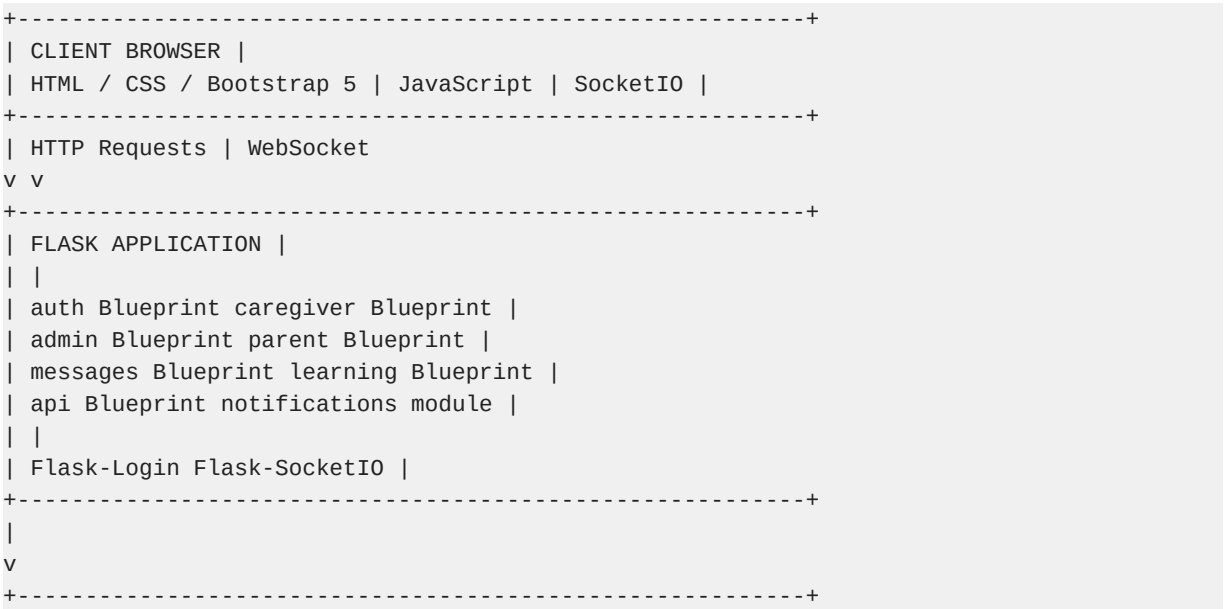
Presentation Tier: The user interface is rendered using Jinja2 HTML templates served by Flask. Bootstrap 5 provides the responsive grid and component framework. Custom CSS in `static/css/style.css` and JavaScript in `static/js/main.js` and `static/js/chat.js` add application-specific behaviour. The Socket.IO JavaScript client library handles WebSocket events on the frontend.

Application Logic Tier: Flask handles all HTTP requests through a set of registered Blueprints. Each Blueprint encapsulates a specific functional area of the application. Business logic — such as determining attendance state from a QR scan, computing monthly attendance statistics, or resolving which user is the appropriate notification recipient — lives in the route handlers and model methods.

Data Tier: Flask-SQLAlchemy manages all database interactions through its ORM. The database schema is defined by Python model classes, each inheriting from `db.Model`. The default database is a SQLite file (`daycare.db`) stored in the project root. All database operations are transactional, and errors trigger a session rollback.

The WebSocket layer, provided by Flask-SocketIO, sits alongside the HTTP layer. Both operate within the same Flask application context, sharing access to the database session and the current user. The SocketIO instance is initialized in the application factory and passed to the notifications module through the `set_socketio` function.

4.2 Application Layer Diagram



```
| FLASK-SQLALCHEMY ORM LAYER |
| User | Child | ActivityLog | Attendance | Message |
| Notification | Photo | LearningResource | Complaint |
+-----------------------------------------------------------+
|
v
+-----------------------------------------------------------+
| SQLite / PostgreSQL |
| daycare.db |
+-----------------------------------------------------------+
```

4.3 Database Design

The database consists of nine tables, each corresponding to a model class defined in the `models` package. The tables and their columns are described below.

Table: users

This table stores all user accounts regardless of role. The `role` column distinguishes between admin, caregiver, and parent accounts.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
name	VARCHAR(120)	NOT NULL	Full name of the user
email	VARCHAR(120)	UNIQUE, NOT NULL, INDEX	Login email address
password_hash	VARCHAR(256)	NOT NULL	Bcrypt password hash
role	VARCHAR(20)	NOT NULL, DEFAULT 'parent'	User role: admin, caregiver, parent
phone	VARCHAR(30)	NULLABLE	Contact phone number
created_at	DATETIME	DEFAULT utcnow	Account creation timestamp
is_approved	BOOLEAN	DEFAULT TRUE	FALSE for pending caregivers
reset_token	VARCHAR(100)	NULLABLE, INDEX	Password reset token
reset_token_expiry	DATETIME	NULLABLE	Token expiration time

Table: children

This table stores profiles of enrolled children. Each child has a mandatory parent and an optional caregiver assignment.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier

Column	Type	Constraints	Description
name	VARCHAR(120)	NOT NULL	Child's full name
dob	DATE	NOT NULL	Date of birth
gender	VARCHAR(20)	DEFAULT 'unspecified'	Gender classification
allergies	TEXT	NULLABLE	Known allergies and dietary notes
notes	TEXT	NULLABLE	General care notes for caregivers
photo_url	VARCHAR(255)	NULLABLE	URL to profile photo
qr_code_path	VARCHAR(255)	NULLABLE	Relative path to QR code image
parent_id	INTEGER	FK users.id, NOT NULL	Parent account reference
caregiver_id	INTEGER	FK users.id, NULLABLE	Assigned caregiver reference
created_at	DATETIME	DEFAULT utcnow	Enrolment timestamp

Table: activity_logs

This table records every activity logged by a caregiver for a child.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
child_id	INTEGER	FK children.id, NOT NULL	Child this log belongs to
type	VARCHAR(30)	NOT NULL	meal, nap, mood, diaper, note, milestone
title	VARCHAR(150)	NOT NULL	Brief summary of the activity
details	TEXT	NULLABLE	Extended description or observations
logged_by	INTEGER	FK users.id, NOT NULL	Caregiver who logged the entry
timestamp	DATETIME	DEFAULT utcnow, INDEX	When the activity occurred

Table: attendance

This table tracks daily attendance for each child.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
child_id	INTEGER	FK children.id, NOT NULL	Child this record belongs to

Column	Type	Constraints	Description
date	DATE	NOT NULL, INDEX	The calendar date of this record
check_in_time	DATETIME	NULLABLE	Timestamp of arrival
check_out_time	DATETIME	NULLABLE	Timestamp of departure
status	VARCHAR(20)	DEFAULT 'present'	present, absent, late, excused
marked_by	INTEGER	FK users.id, NOT NULL	Staff who marked the record
notes	TEXT	NULLABLE	Attendance notes

Table: messages

This table stores all messages in the parent-caregiver chat system.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
child_id	INTEGER	FK children.id, NOT NULL	Child this message concerns
sender_id	INTEGER	FK users.id, NOT NULL	User who sent the message
receiver_id	INTEGER	FK users.id, NOT NULL	Intended recipient
content	TEXT	NOT NULL	Message body
timestamp	DATETIME	DEFAULT utcnow, INDEX	Send timestamp
is_alert	BOOLEAN	DEFAULT FALSE	Whether this is an emergency alert
alert_severity	VARCHAR(20)	DEFAULT 'info'	info, warning, critical
is_read	BOOLEAN	DEFAULT FALSE	Read receipt flag

Table: notifications

This table stores in-app notifications for users.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
user_id	INTEGER	FK users.id CASCADE DELETE	Recipient user
title	VARCHAR(150)	NOT NULL	Short notification headline
message	TEXT	NOT NULL	Full notification body
link	VARCHAR(255)	NULLABLE	URL to navigate to on click

Column	Type	Constraints	Description
is_read	BOOLEAN	DEFAULT FALSE, INDEX	Whether notification has been read
is_urgent	BOOLEAN	DEFAULT FALSE	Triggers SMS/email simulation
created_at	DATETIME	DEFAULT utcnow, INDEX	Creation timestamp

Table: photos

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
child_id	INTEGER	FK children.id, NOT NULL	Child this photo depicts
filepath	VARCHAR(255)	NOT NULL	Relative path under static/
caption	TEXT	NULLABLE	Photo description
category	VARCHAR(50)	DEFAULT 'Activity'	Art, Outdoor, Meal, Milestone, Nap, Group
uploaded_by	INTEGER	FK users.id, NOT NULL	Caregiver who uploaded
timestamp	DATETIME	DEFAULT utcnow, INDEX	Upload timestamp

Table: learning_resources

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
title	VARCHAR(150)	NOT NULL	Resource name
type	VARCHAR(30)	NOT NULL	rhyme, story, activity, game
age_group	VARCHAR(30)	NOT NULL	infant, toddler, preschool, all
category	VARCHAR(50)	DEFAULT 'General'	Thematic category label
description	TEXT	NULLABLE	Brief overview
content_url_or_text	TEXT	NOT NULL	Full content body or URL
icon	VARCHAR(50)	DEFAULT 'bi-book'	Bootstrap icon class
created_at	DATETIME	DEFAULT utcnow	Creation timestamp

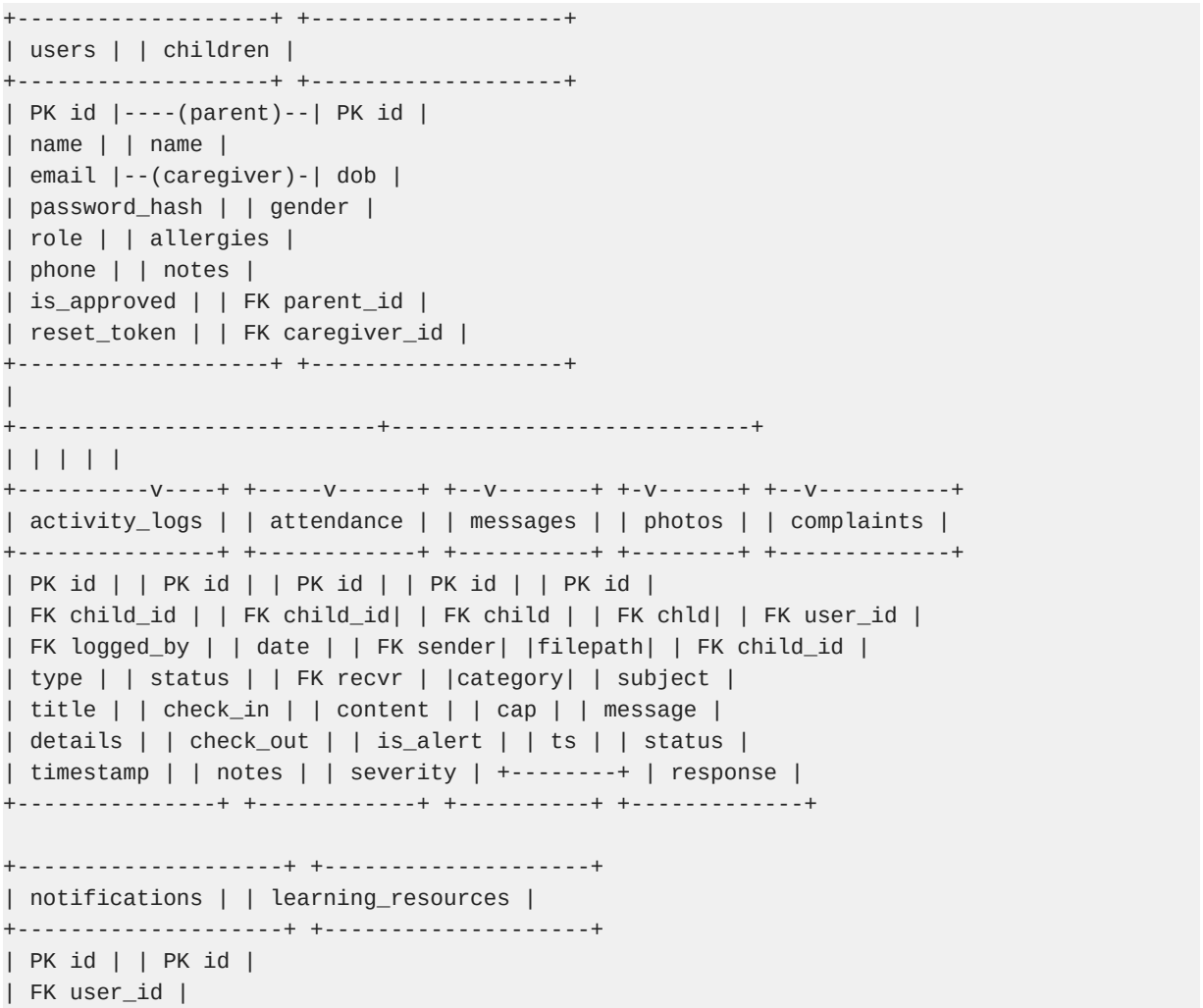
Table: complaints

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-increment identifier
user_id	INTEGER	FK users.id, NOT NULL	Parent who submitted

Column	Type	Constraints	Description
child_id	INTEGER	FK children.id, NULLABLE	Related child (optional)
subject	VARCHAR(200)	NOT NULL	Brief subject line
message	TEXT	NOT NULL	Full complaint or inquiry text
status	VARCHAR(30)	DEFAULT 'open'	open, in_progress, resolved
admin_response	TEXT	NULLABLE	Director's written response
created_at	DATETIME	DEFAULT utcnow	Submission timestamp
updated_at	DATETIME	DEFAULT utcnow, ON UPDATE	Last modification timestamp

---

4.4 Entity Relationship Diagram



```
| is_urgent | | content_url_text |
| created_at | | icon |
+--------------------+ +--------------------+
```

4.5 Use Case Diagram

The following describes the use cases for each actor in the system.

```
+----------------------------+
| Virtual Daycare System |
| |
ADMIN | Approve Caregiver |
o | Reject Caregiver |
/|\ ----(manages)---> | Enrol Child |
/ \ | Edit Child Profile |
| View Dashboard Stats |
| Manage Learning Library |
| Respond to Complaints |
| |
CAREGIVER | View Assigned Children |
o | Log Activity |
/|\ ----(records)---> | Upload Photo |
/ \ | Mark Attendance |
| Scan QR Code |
| Send Emergency Alert |
| View and Send Messages |
| |
PARENT | View Child Dashboard |
o | View Activity Timeline |
/|\ ----(monitors)--> | Browse Photo Gallery |
/ \ | View Attendance Records |
| View Child Profile |
| Submit Complaint |
| View and Send Messages |
| |
ALL AUTHENTICATED | Register / Login / Logout |
USERS | Reset Password |
| View Notifications |
| Browse Learning Library |
+----------------------------+
```

4.6 Class Diagram

```
+---------------------------------+
| User |
+---------------------------------+
| - id: int |
| - name: str |
| - email: str |
| - password_hash: str |
| - role: str |
| - phone: str |
```

```
| - is_approved: bool |
| - reset_token: str |
| + set_password(password) |
| + check_password(password): bool |
| + generate_reset_token(): str |
| + verify_reset_token(token): bool|
| + clear_reset_token() |
| @ is_admin: bool |
| @ is_caretaker: bool |
| @ is_parent: bool |
+----------------------------------+
1 1
| |
| parent | caregiver
| |
n n
+----------------------------------+
| Child |
+----------------------------------+
| - id: int |
| - name: str |
| - dob: date |
| - gender: str |
| - allergies: str |
| - notes: str |
| - parent_id: int (FK) |
| - caregiver_id: int (FK) |
| + generate_qr_code(dir) |
| @ age_string: str |
| @ age_group: str |
| @ today_attendance: Attendance |
| @ is_checked_in: bool |
| @ is_checked_out: bool |
| @ active_emergency_alert: Message|
| + get_monthly_attendance_stats() |
+----------------------------------+
| | | | |
| | | | |
ActivityLog Attendance Message Photo Complaint
```

4.7 Sequence Diagrams

Sequence 1: Caregiver Logs Activity — Parent Receives Notification

```
Caregiver Flask Route Database Notifications Parent Browser
| | | | |
|--POST /log------->| | | |
| |--INSERT log----->| | |
| |<--log saved------| | |
| |--send_notif()--->| | |
| | |--INSERT notif->| |
| | | |--emit WebSocket->|
| |--redirect------->| | |
|<--Caregiver Page--| | | |
```

Sequence 2: QR Code Scan — Check-In

```
Caregiver Browser /api/qr-scan Database Parent Browser
| | | |
|--POST {qr_data}-->| | |
| |--Parse CHILD_ID->| |
| |--Query Attendance| |
| |<--None (no rec)--| |
| |--INSERT check_in>| |
| |--notify_parents()> |
| | | |<--WebSocket event
|<--{check_in JSON}-| | |
```

Sequence 3: Emergency Alert Dispatch

```
Caregiver /caregiver/alert Database Notifications Parent
| | | | |
|--POST /alert---->| | | |
| |--INSERT Message-->| | |
| | (is_alert=True) | | |
| |--notify(urgent)-->| | |
| | |--INSERT notif>| |
| | | |--WebSocket-->|
| | | |--send_sms()-->[log]
| | | |--send_email()->[log]
|<--Redirect+Flash-| | | |
```

Sequence 4: Password Reset Flow

```
Parent Browser /reset-password Database
| | |
|--POST {email}-------->| |
| |--Query User------->|
| |<--User object------|
| |--generate_token() |
| |--UPDATE user------->|
|<--Page with link------| |
| | |
|--POST {token, pw}---->| |
| |--verify_token() |
| |--set_password() |
| |--clear_token()---->|
|<--Redirect to login---| |
```

4.8 Data Flow Diagram

Level 0 — Context Diagram

```
+--------------------------+
PARENT ---data---> | |
(queries, messages) | Virtual Daycare | <--- CAREGIVER
| Platform System | (logs, photos,
PARENT <---data--- | | attendance, alerts)
```

```
(notifications, | |
timeline, photos) +---------------------------+
^
|
ADMIN
(enrol children, approve
caregivers, manage library,
respond to complaints)
```

Level 1 — Primary Data Flows

1. User registration data flows through the Auth module to create a User record. Caregiver registration triggers admin notification.

2. Caregiver activity form data flows through the Caregiver module to create an ActivityLog record, which triggers a parent notification via the Notifications module.

3. QR scan data flows through the API module to query and update the Attendance record, triggering a parent notification.

4. Parent message submission flows through the Messages module to create a Message record, emitting a WebSocket event and creating a Notification record for the receiver.

5. Admin complaint response flows through the Admin module to update the Complaint record and create a Notification for the parent.

Chapter 5: Technology Stack

5.1 Backend Framework — Flask

Flask is a lightweight WSGI web framework for Python. Unlike Django, which follows a batteries-included approach, Flask is a microframework that provides the essential components of a web framework — request routing, templating, and request context — while leaving the developer free to choose their own tools for the remaining concerns.

Flask was chosen for this project because the application needs tight control over its structure, which the Blueprint architecture provides cleanly. Flask-SQLAlchemy, Flask-Login, and Flask-SocketIO are all first-class Flask extensions that integrate seamlessly. Flask's application factory pattern allows the application to be instantiated with different configurations for testing and production without any conflict. Flask is also sufficiently lightweight to run comfortably on small cloud instances.

The application is initialized in `app.py` using the `create_app(config_class)` factory function. This function creates the Flask instance, loads configuration, initializes extensions, registers blueprints, and defines context processors and SocketIO event handlers.

5.2 Database — SQLAlchemy with SQLite

Flask-SQLAlchemy is the ORM extension for Flask that wraps SQLAlchemy, one of the most capable Python database toolkits available. ORM maps Python class definitions to database tables, allowing developers to interact with the database using Python objects rather than writing raw SQL.

SQLite is used as the default database for development and testing. SQLite stores the entire database as a single file on disk, requires no server process, and is bundled with Python's standard library, making it ideal for a self-contained development and demonstration environment.

The database URI is configured through the `SQLALCHEMY_DATABASE_URI` environment variable, defaulting to `sqlite:///daycare.db`. Changing this single variable to a PostgreSQL connection string is all that is needed to switch the database backend.

SQLAlchemy's ORM provides several important benefits used in this project. Relationship definitions allow related objects to be accessed naturally through Python attributes — for example, `child.parent` returns the User object for the child's parent. Lazy loading with `lazy='dynamic'` on collection relationships returns a query object rather than loading all records immediately. Cascade operations on relationships such as `cascade='all, delete-orphan'` ensure that related records are automatically deleted when a child profile is removed.

5.3 Real-Time Communication — Flask-SocketIO

Flask-SocketIO is the Flask integration of the Socket.IO protocol, which provides a bidirectional event-based communication channel between the browser and the server. Socket.IO builds on WebSockets but also provides automatic fallback to HTTP long polling for environments where WebSocket connections are unavailable.

The SocketIO instance is initialized in the `create_app` function and stored in the `notifications.py` module through the `set_socketio` call. This allows the notifications module to emit events without needing a direct reference to the Flask application object.

Three key SocketIO events are handled on the server. The `join_user_notifications` event places authenticated users into a personal notification room named `user_{user_id}`. The `join_child_chat` event places users viewing a chat thread into a child-specific room named `child_{child_id}`. The `send_chat_message` event allows the browser to send a message through the WebSocket connection; the handler saves the message to the database and broadcasts it to the child room.

5.4 Authentication — Flask-Login

Flask-Login manages user session persistence. When a user logs in, Flask-Login stores the user's ID in the session cookie. On subsequent requests, the `@login_required` decorator and the `current_user` proxy enforce authentication and provide access to the authenticated user's data.

The `User` model inherits from both `UserMixin` (provided by Flask-Login) and `db.Model` (provided by Flask-SQLAlchemy). `UserMixin` provides default implementations of the four Flask-Login required interface methods: `is_authenticated`, `is_active`, `is_anonymous`, and `get_id`.

The `user_loader` callback registered in `create_app` tells Flask-Login how to reconstruct a User object from the ID stored in the session. The `login_view` is set to `auth.login`, which means any protected route accessed without an active session redirects the user to the login page with a warning flash message.

5.5 QR Code Generation — qrcode and Pillow

The `qrcode` library generates QR code images in Python. The `Child.generate_qr_code` method creates a QR code encoding the string `DAYCARE_CHECKIN:CHILD_ID={child.id}` and saves it as a PNG file under `static/uploads/qrcodes/`. The QR code parameters — version 1, box size 10, border 2 — produce a compact but clearly scannable image.

`Pillow` (PIL) is used in the `seed.py` database seeder to generate placeholder activity photos using programmatic image drawing. In production use, real photographs are uploaded by caregivers through the web upload form.

5.6 Frontend Technologies

Bootstrap 5 provides the responsive CSS and JavaScript component framework used for the grid layout, navigation, modals, forms, alerts, badges, and cards across all templates.

Bootstrap Icons is an icon font library used for activity type icons, navigation icons, and status indicators throughout the interface. Each activity type maps to a distinct Bootstrap icon class stored in the model's `icon_class` property.

Jinja2 is Flask's built-in templating engine. All HTML pages are Jinja2 templates that extend the `base.html` layout template. Template inheritance avoids repeating common elements such as the navigation bar, notification panel, and footer.

Socket.IO JavaScript Client is included in templates requiring real-time features. The `chat.js` file manages the child-specific chat room connection and real-time message rendering. The `main.js` file manages the user notification room connection and in-page notification badge updates.

Vanilla JavaScript handles QR code scanning through the browser camera API, dynamic form behaviour based on activity type selection, and AJAX message sending.

5.7 Deployment — Gunicorn and Procfile

For production deployment, the application uses Gunicorn as the WSGI server. Gunicorn can manage multiple worker processes, making it suitable for handling concurrent requests.

The `Procfile` at the project root defines the start command for PaaS platforms such as Heroku or Render:

```
web: gunicorn --worker-class eventlet -w 1 wsgi:app
```

The `eventlet` worker class is required for Flask-SocketIO's WebSocket support to function correctly under Gunicorn. The single worker is required because SocketIO state is not shared between separate worker processes without a Redis message queue backend.

The `wsgi.py` file exports the Flask application instance. The application reads all sensitive configuration from environment variables defined in the `.env` file. The `.env.example` file documents all required environment variables with placeholder values.

Chapter 6: Module Description

6.1 Authentication Module

File: `routes/auth.py`

Blueprint: `auth_bp` (no URL prefix)

Templates: `templates/auth/`

The authentication module handles all aspects of user account management including registration, login, logout, and password reset.

Registration — POST /register

The registration form collects the user's full name, email address, password, password confirmation, role, and phone number. Parents registering through this form may optionally enter their child's name and date of birth, which creates a basic child profile linked to the new parent account with an auto-generated QR code.

Validation rules enforced on submission:

- All three required fields must be present
- Password must be at least 6 characters long
- Password and confirmation must match
- Role must be one of: parent, caregiver, caretaker
- Email must not already exist in the database

For parents, the account is immediately active. For caregivers, the `is_approved` flag is set to `False`, and all administrators receive a notification about the pending registration.

Login — POST /login

The login form accepts email and password. Email is normalized to lowercase before querying. If credentials are valid, the user is logged in using Flask-Login's `login_user`. Caregivers with pending approval are logged in but immediately redirected to the pending approval page. Post-login, a `next` parameter in the query string is respected to redirect to the originally requested page.

Logout — GET /logout

Calls `logout_user()` to clear the session, then redirects to the login page with an informational flash message.

Pending Approval — GET /pending-approval

Caregivers are redirected here after login if their account has not been approved. This route is only accessible to unapproved caregivers — approved users are redirected to their dashboard.

Password Reset Request — POST /reset-password

The user enters their registered email address. If found, `user.generate_reset_token()` is called, which creates a cryptographically secure 32-byte URL-safe token using `secrets.token_urlsafe(32)` and sets a one-hour expiry. Because email service is not configured in this demonstration setup, the reset link is displayed directly on the page with a note indicating this is simulated delivery.

Password Reset — POST /reset-password/<token>

The token is looked up in the database. If not found or expired, the user is redirected to the reset request page. If valid, the new password is hashed and saved, and the reset token is cleared.

6.2 Administrator Module

File: `routes/admin.py`

Blueprint: `admin_bp` (URL prefix: `/admin`)

Templates: `templates/admin/`

Every route in this blueprint is protected by both `@login_required` and `@role_required('admin')` through the `before_request` hook. Non-admin users receive a 403 Forbidden response.

Dashboard — GET /admin/dashboard

Aggregates real-time metrics for the director:

- `total_children` — Total count of all enrolled children
- `active_caregivers` — Count of approved caregivers
- `pending_caregivers` — Count of awaiting-approval caregivers
- `total_parents` — Count of all parent-role accounts
- `open_complaints` — Count of complaints with status 'open'
- `attendance_pct` — Percentage of children marked present or late today
- `recent_logs` — The 10 most recently created activity log entries
- `recent_complaints` — The 5 most recently submitted complaints

Caregiver Management — GET /admin/caregivers

Lists all caregiver accounts sorted by approval status ascending (pending first) then by creation date descending, so pending caregivers appear at the top for immediate attention.

Approve Caregiver — POST /admin/approve/<user_id>

Sets the caregiver's `is_approved` flag to `True` and creates a notification for the caregiver informing them their account is now active with a link to the caregiver dashboard.

Reject Caregiver — POST /admin/reject/<user_id>

Sets the caregiver's `is_approved` flag to `False`. This can both initially deny a caregiver and revoke a previously granted approval.

Children Management — GET /admin/children

Lists all enrolled children alphabetically by name. Each child's record shows their name, computed age, assigned parent, assigned caregiver, and an edit button.

Enrol New Child — POST /admin/child/new

The enrollment form requires the child's name, date of birth, and parent assignment. Gender, caregiver assignment, allergies, and notes are optional. On submission, the child record is created, the parent receives an enrollment notification, and QR code generation is handled during subsequent editing if not triggered at creation.

Edit Child Profile — POST /admin/child/<child_id>/edit

Allows updating any field of an existing child profile. If the child does not yet have a QR code path, it is generated during this operation.

Complaints Management — GET /admin/complaints

Lists all complaints with optional status filtering through the `status` query parameter. Filtering options are: all, open, in_progress, resolved.

Respond to Complaint — POST /admin/complaint/<complaint_id>/respond

The administrator enters a response text and selects a new status. The complaint record is updated and a notification is sent to the submitting parent.

Learning Resource Management

Three operations are provided:

- `GET /admin/learning` — Lists all resources ordered by creation date descending
- `POST /admin/learning/new` — Creates a new resource with all required fields
- `POST /admin/learning/<id>/edit` — Updates an existing resource
- `POST /admin/learning/<id>/delete` — Deletes the resource and redirects to the list

6.3 Caregiver Module

File: `routes/caregiver.py`

Blueprint: `caregiver_bp` (URL prefix: `/caregiver`)

Templates: `templates/caregiver/`

All routes are protected by `@login_required` and `@role_required('caregiver', 'admin')`. Admin users can access caregiver routes and see all children, while caregivers are restricted to their assigned children.

Dashboard — GET /caregiver/dashboard

Admins see all children. Caregivers see only children where `caregiver_id == current_user.id`. Each child is rendered as a card with a summary of their current status for the day.

Child Detail — GET /caregiver/child/<child_id>

The primary working page for a caregiver managing an individual child. Access is validated by `get_authorized_child` first. The page displays today's activity logs in reverse chronological order, the most recent 6 photos uploaded for this child, today's attendance record with timestamps, and quick action buttons for all caregiver operations.

Log Activity — POST /caregiver/child/<child_id>/log

The activity logging form adapts its visible input fields based on the selected activity type using JavaScript. Field sets for each type are:

- **Meal:** Meal type (Breakfast, Lunch, Afternoon Snack, Dinner), food items, amount eaten, notes
- **Nap:** Start time, end time, nap quality (Peaceful, Restless, Did Not Sleep), notes
- **Mood:** Mood value (Happy, Playful, Tired, Fussy, Upset, Calm, Excited), notes
- **Diaper:** Type (Wet, Soiled, Dry Check), notes
- **Milestone:** Title and description of the achievement
- **Note:** Free-text title and details

An optional custom timestamp field allows backdating an entry if it was not logged immediately at the time of the activity. After saving, a notification is sent to the child's parent linking to the activity timeline.

Upload Photo — POST /caregiver/child/<child_id>/photo

The upload form accepts an image file along with a caption and a category selection (Art, Outdoor, Meal, Milestone, Nap, Activity, Group). File validation checks the extension and size. A UUID-prefixed unique filename is generated to prevent collisions. After upload, the parent receives a notification with a link to the photo gallery.

Mark Attendance — POST /caregiver/child/<child_id>/attendance

Four actions are supported through the `action` form field:

- **check_in** — Sets `check_in_time` to the current server time, sets status to 'present', and notifies the parent.
- **check_out** — Sets `check_out_time` to the current server time and notifies the parent.
- **mark_absent** — Sets status to 'absent' and records an optional reason note.
- **manual_update** — Allows setting specific check-in and check-out times and an overall status, useful for correcting records or entering data after the fact.

If no attendance record exists for today, a new one is created. If one already exists, it is updated.

Daily Attendance Roster — GET /caregiver/attendance

Shows all assigned children in a roster layout with their current attendance status for today. Quick check-in and check-out buttons allow processing the morning arrival of multiple children without navigating away from the roster page.

Emergency Alert — POST /caregiver/child/<child_id>/alert

The emergency alert form takes a title, description, and severity level (info, warning, critical). On submission:

1. A `Message` record is created with `is_alert=True` and the chosen severity. The content is prefixed with the alert title in uppercase brackets for immediate visual identification.
2. `notify_child_parents` is called with `is_urgent=True`, creating a Notification record and triggering both the WebSocket emit and the simulated SMS and email dispatch.
3. The alert appears in the parent's dashboard as a prominently styled alert banner distinct from regular notifications.

QR Scanner — GET /caregiver/qr-scanner

Redirects to the daily attendance roster. QR processing is handled by the API module at `POST /api/qr-scan`.

6.4 Parent Module

File: `routes/parent.py`

Blueprint: `parent_bp` (URL prefix: `/parent`)

Templates: `templates/parent/`

All routes are protected by `@login_required` and `@role_required('parent', 'admin')`. Admin users can access the parent view for all children. Regular parents can only access data for their own children, enforced by `get_authorized_child`.

Dashboard — GET /parent/dashboard

For each child belonging to the current parent, the dashboard gathers: all activity logs recorded today, the most recent photo uploaded, the current active emergency alert (if any for today), and today's attendance record. This data is bundled into a list of dictionaries — one per child — and passed to the template.

Emergency alerts are surfaced as high-visibility banners at the top of each child's card. The alert banner includes the full message text, timestamp, and severity styling so the parent cannot overlook a health or safety notification.

Activity Timeline — GET /parent/child/<child_id>/timeline

Shows all activity logs and photos for a specific child on a selected date. The date is controlled by a `date` query parameter in `YYYY-MM-DD` format; today is used if not provided. Logs and photos are displayed in chronological order, combining both types into a unified timeline. The attendance record for the selected date is also shown. Parents navigate between dates using a date picker input.

Photo Gallery — GET /parent/child/<child_id>/photos

Lists all photos uploaded for the child, ordered by timestamp descending. Photos can be filtered by category using the `category` query parameter. Available categories are All, Activity, Outdoor, Art, Meal, Milestone, and Nap.

Attendance Records — GET /parent/child/<child_id>/attendance

The attendance page shows a monthly summary computed by `child.get_monthly_attendance_stats(year, month)`, which calculates the number of weekdays in the month, counts present or late days, counts absent days, and computes the percentage. Below the summary, a detailed table of the 30 most recent attendance records shows date, status badge, check-in time, check-out time, duration, and notes. Year and month can be adjusted through query parameters for reviewing past months.

Child Profile — GET /parent/child/<child_id>/profile

Shows the child's complete profile information: name, date of birth, calculated age, gender, assigned caregiver, allergies, special notes, and the QR code image for use at drop-off.

Complaints and Feedback — POST /parent/complaints

Parents submit inquiries or feedback through a form collecting a subject line, message body, and an optional child association. On submission, the complaint is saved and all admin users receive a notification. The GET view of this page lists all complaints previously submitted by the current parent with their current status badge and any admin response.

6.5 Messaging Module

File: `routes/messages.py`

Blueprint: `messages_bp` (no URL prefix)

Templates: `templates/messages/`

The messaging system implements private, child-centric communication between parents and caregivers.

Thread List — GET /messages

Shows all conversation threads accessible to the current user — one per child. For each thread, the page shows the child's name, the most recent message preview, and the unread message count for the current user.

Chat Thread — GET/POST /messages/<child_id>

The chat interface for a specific child's conversation. The receiver is determined by role: the parent messages the assigned caregiver; the caregiver messages the parent; the admin messages the parent. When the page loads, all unread messages directed to the current user in this thread are marked as read.

POST requests send a new message. The message is saved, a notification is created for the receiver, and the user is redirected back to the thread. The `chat.js` file connects to SocketIO, joins the child's chat room, and listens for `new_chat_message` events to render incoming messages in real time without a page refresh.

API Endpoints

`GET /api/messages/<child_id>` — Returns all messages for a child's thread as JSON. Used by the chat JavaScript for polling-based refresh. Also marks incoming messages as read.

`POST /api/messages/<child_id>/send` — Accepts a JSON payload, creates the message, sends a notification to the receiver, and returns the new message as JSON. Used for AJAX-based sending without page reload.

6.6 Learning Resources Module

File: `routes/learning.py`

Blueprint: `learning_bp` (URL prefix: `/learning`)

Templates: `templates/learning/`

All routes require authentication but are accessible to all three user roles.

Library Index — GET /learning/

Displays all learning resources with three optional filter parameters: `age` (infant, toddler, preschool, all), `type` (rhyme, story, activity, game), and `q` (full-text search against title, description, and category using case-insensitive LIKE queries). When an age group filter is selected, resources tagged 'all' also appear in the results since they are universally appropriate.

Resource Detail — GET /learning/<resource_id>

Displays the full content of a selected resource. Related resources of the same type (limited to 3) are shown as suggestions.

The default seeded library contains 8 resources covering all four types and all age groups including: The Itsy Bitsy Spider (rhyme, infant), Twinkle Twinkle Little Star (rhyme, all ages), The Little Red Hen (story, toddler), The Three Little Pigs (story, preschool), Sensory Ice Exploration (activity, toddler), Rainbow Color Sorting (activity, preschool), Tummy Time Mirror Exploration (activity, infant), and Simon Says Animal Edition (game, preschool).

6.7 Notifications Module

File: `notifications.py`

Not a Blueprint — standalone utility module

The notifications module centralizes all notification dispatch logic and maintains a module-level reference to the SocketIO instance set during application startup.

send_notification(user_id, title, message, link, is_urgent, emit_socket)

The core dispatch function. Steps:

1. Creates a `Notification` record in the database.
2. If `emit_socket` is True and the SocketIO instance is available, emits a `new_notification` WebSocket event to `user_{user_id}` room.
3. If `is_urgent` is True, looks up the user's phone and email and calls the `send_sms` and `send_email` hooks.
4. Returns the Notification object, or None if an error occurred (after rolling back the database session).

notify_child_parents(child, title, message, link, is_urgent)

Convenience wrapper that calls `send_notification` for the child's `parent_id`. Safe to call even if the child has no parent — it checks before dispatching.

notify_caregiver(child, title, message, link, is_urgent)

Convenience wrapper that calls `send_notification` for the child's `caregiver_id`.

notify_all_admins(title, message, link, is_urgent)

Queries all users with `role='admin'` and sends each one a notification. Used for caregiver registration alerts and parent complaint submissions.

send_sms(to_phone, message_text) and send_email(to_email, subject, body_html)

Production hook functions. In the current implementation they log the dispatch to the Python logger and print to console. In production, these would be replaced with Twilio and SendGrid API calls while the function signatures remain unchanged.

6.8 REST API Module

File: `routes/api.py`

Blueprint: `api_bp` (URL prefix: `/api`)

The API module exposes JSON endpoints used by the frontend JavaScript and the QR scanner.

GET /api/notifications/unread

Returns the total count of unread notifications and an array of the 10 most recent unread notifications as dictionaries. Polled by the notification JavaScript to keep the navigation badge current.

POST /api/notifications/mark-read/<notif_id>

Marks a single notification as read after verifying it belongs to the current user. Returns `{success: true}`.

POST /api/notifications/mark-all-read

Batch-updates all unread notifications for the current user to `is_read=True` using SQLAlchemy's bulk update.

POST /api/qr-scan

The most operationally significant API endpoint. Accepts a JSON body with a `qr_data` field. Processing steps:

1. Verifies the current user is a caregiver or admin.
2. Parses the child ID from the QR string using the regex `CHILD_ID=(\d+)`.
3. Fetches and authorizes the child.
4. Queries today's attendance record for the child.
5. If no record exists, creates a check-in and notifies the parent.
6. If check-in exists but no check-out, updates the check-out time and notifies the parent.
7. If both exist, returns an informational already-completed message.

The response in all cases includes a `success` boolean, an `action` string (check_in, check_out, or already_completed), the child's name, the timestamp, and a human-readable message for display in the scanner interface.

Chapter 7: Database Schema and Models

7.1 User Model

File: `models/user.py` — **Table:** `users`

The User model is the central identity record for all system participants. Three boolean properties — `is_admin`, `is_caretaker`, and `is_parent` — evaluate the `role` string and return booleans, making role checks throughout the codebase readable and consistent. The `is_caretaker` property accepts both 'caretaker' and 'caregiver' as valid role strings for backward compatibility.

Password management uses two methods. `set_password(password)` hashes the plain-text password using Werkzeug's `generate_password_hash` and stores the result. `check_password(password)` uses `check_password_hash` to compare the provided plain text against the stored hash. Passwords are never stored in retrievable form.

The password reset mechanism uses `secrets.token_urlsafe(32)`, generating a URL-safe base64-encoded string from 32 random bytes — approximately 43 characters of cryptographically secure randomness. The token and its expiry (current UTC time plus one hour) are saved to the user record. `verify_reset_token` checks the token matches and has not expired.

The `display_role` property returns a human-readable role label, mapping 'caretaker' or 'caregiver' to "Caretaker" and using `.title()` on other roles.

7.2 Child Model

File: `models/child.py` — **Table:** `children`

The Child model is the central operational entity. Every activity log, photo, attendance record, message, and complaint can reference a specific child.

The model defines several computed properties. `age_string` calculates the child's precise age from date of birth to today, returning strings such as "2 yrs 3 mo" or "8 months", handling sub-month precision and pluralization edge cases. `age_group` returns 'infant' for children under 1 year, 'toddler' for 1 to 3 years, and 'preschool' for 3 years and older, used for matching learning resources to the child.

`today_attendance` queries the Attendance table for a record matching today's date for this child. `is_checked_in` returns True if today's record has a check-in time but no check-out time. `is_checked_out` returns True if both times are present. `active_emergency_alert` queries the Message table for the most recent alert message today with `is_alert=True`, used by the parent dashboard to surface health warnings.

`generate_qr_code(qrcode_dir)` uses the `qrcode` library to generate a QR code encoding `DAYCARE_CHECKIN:CHILD_ID={self.id}` and saves it as a PNG.

`get_monthly_attendance_stats(year, month)` counts weekdays in the target month up to today (for the current month) or to the last day (for past months), then counts present/late records and computes the percentage. Returns a dictionary with `total_possible_days`, `days_present`, `days_absent`, `percentage`, `month_name`, and `year`.

7.3 ActivityLog Model

File: `models/activity.py` — **Table:** `activity_logs`

The ActivityLog model records individual care events. The six activity types each have distinct operational semantics. Meal records what was served and how much was eaten. Nap records the sleep window and quality assessment. Mood records the child's emotional state. Diaper records changes and skin condition notes. Note is a free-form observation entry. Milestone records a developmental achievement.

The `icon_class` property returns a Bootstrap icon and colour class string appropriate to each activity type. The `badge_class` property returns a CSS badge class for visual differentiation in the timeline view. The `formatted_time` and `formatted_date` properties return human-readable timestamp strings.

7.4 Attendance Model

File: `models/attendance.py` — **Table:** `attendance`

Each row represents one child's attendance for one calendar day. The four statuses serve different operational purposes. `present` is normal attendance with check-in and check-out times populated as they occur. `absent` means the child did not attend and times remain null. `late` means the child arrived after the normal start time but still attended. `excused` is an absence for a documented reason.

The `duration_str` property computes total time at the daycare by subtracting `check_in_time` from `check_out_time` and converting to hours and minutes. If only check-in exists, it returns "Checked in (In

Daycare)". The `badge_class` property returns Bootstrap colour classes for the status: green for present, red for absent, yellow for late, and teal for excused.

7.5 Message Model

File: `models/message.py` — **Table:** `messages`

Messages link a child, a sender, and a receiver. The `is_alert` flag distinguishes emergency alerts from regular messages. When `is_alert` is True, the `alert_severity` field (info, warning, critical) provides context for visual styling and priority level in the interface.

The `to_dict()` method serializes the message to a Python dictionary for JSON responses in the API and WebSocket events. It resolves the sender and receiver name and role from the relationship objects, with fallback strings if the related user has been deleted.

7.6 Notification Model

File: `models/notification.py` — **Table:** `notifications`

Notifications are user-targeted records created by system events. The `is_read` field is indexed for fast filtering, queried on every authenticated page load for the navigation badge count. The `is_urgent` flag determines whether external dispatch is triggered.

The `formatted_time` property returns a human-readable relative time: "Just now", "5m ago", "2h ago", or "3d ago". The `to_dict()` method serializes the notification for the WebSocket payload and the unread notifications JSON API response.

7.7 Photo Model

File: `models/photo.py` — **Table:** `photos`

The Photo model stores metadata about uploaded child photographs. The actual image file is stored on disk under `static/uploads/photos/`, and `filepath` stores the relative path from the static directory. Flask serves the file at the corresponding URL when the template renders it.

The seven categories — Activity, Outdoor, Art, Meal, Milestone, Nap, Group — allow parents to filter the photo gallery. The category is set by the caregiver at upload time using a dropdown selection.

7.8 LearningResource Model

File: `models/learning.py` — **Table:** `learning_resources`

The `content_url_or_text` Text column holds either a URL reference to an external resource or the full text of the content. The default seed data uses full-text content for all eight seeded resources.

The `type_badge_class` property returns Bootstrap background colour classes by content type: pink for rhymes, primary blue for stories, success green for activities, and warning yellow for games. The `age_group_display` property returns the full human-readable label: "Infant (0-1 yr)", "Toddler (1-3 yrs)", "Preschool (3-5 yrs)", or "All Ages".

7.9 Complaint Model

File: `models/complaint.py` — **Table:** `complaints`

The three status values progress from `open` (newly submitted) through `in_progress` (administrator acknowledged) to `resolved` (director has responded and closed the matter).

The `admin_response` field stores the director's written reply displayed to the parent alongside the status badge. The `updated_at` column is configured with `onupdate=datetime.utcnow`, so SQLAlchemy automatically updates this timestamp whenever the record is modified. The `badge_class` property returns colour-coded Bootstrap classes matching each status.

Chapter 8: Implementation

8.1 Project Structure

```
virtual-daycare/
■■■ app.py Application factory and SocketIO event handlers
■■■ config.py Configuration class
■■■ notifications.py Notification dispatch module
■■■ wsgi.py WSGI entry point for Gunicorn
■■■ init_db.py Production database initialization
■■■ seed.py Development seed data generator
■■■ test_daycare.py Test suite
■■■ requirements.txt Python package dependencies
■■■ Procfile Deployment command
■■■ .env Environment variables (not in version control)
■■■ .env.example Environment variable template
■
■■■ models/
■ ■■■ __init__.py SQLAlchemy instance and model imports
■ ■■■ user.py
■ ■■■ child.py
■ ■■■ activity.py
■ ■■■ attendance.py
■ ■■■ message.py
■ ■■■ notification.py
■ ■■■ photo.py
■ ■■■ learning.py
■ ■■■ complaint.py
■
■■■ routes/
■ ■■■ __init__.py Blueprint aggregator
■ ■■■ auth.py
■ ■■■ admin.py
```

```
■ ■■■ caregiver.py
■ ■■■ parent.py
■ ■■■ messages.py
■ ■■■ learning.py
■ ■■■ api.py
■ ■■■ utils.py
■
■■■ templates/
■ ■■■ base.html
■ ■■■ landing.html
■ ■■■ auth/ login, register, reset templates
■ ■■■ admin/ dashboard, caregivers, children, learning templates
■ ■■■ caregiver/ dashboard, child detail, activity, attendance templates
■ ■■■ parent/ dashboard, timeline, photos, attendance, complaints
■ ■■■ messages/ list and thread templates
■ ■■■ learning/ index and detail templates
■ ■■■ 403.html, 404.html, 500.html
■
■■■ static/
■■■ css/style.css
■■■ js/main.js
■■■ js/chat.js
■■■ uploads/
■■■ photos/
■■■ qrcodes/
```

8.2 Application Factory Pattern

The application uses Flask's Application Factory pattern — the Flask application is not created at module level but inside the `create_app(config_class)` function. This provides two critical benefits.

First, it makes the application fully testable. The test suite calls `create_app(TestConfig)` with a configuration class that sets the database URI to an in-memory SQLite instance. Without the factory pattern, the production configuration would be locked in at import time.

Second, it prevents circular imports. The `db` instance, the SocketIO instance, and the `login_manager` are all created inside `create_app` and bound to the app instance there. Models and routes access `db` through the `models` package's `__init__.py`, not through the application module.

The function returns both the Flask app and the SocketIO instance because running the development server requires `socketio.run(app)` rather than `app.run()` to enable WebSocket support.

8.3 Configuration Management

The `Config` class in `config.py` uses `os.environ.get(key, default)` to read every setting from environment variables, falling back to development defaults when variables are absent.

`python-dotenv` reads the `.env` file at startup and populates `os.environ`. The context processor in `create_app` injects `daycare_name`, `daycare_phone`, and `daycare_email` into every template:

```
@app.context_processor
def inject_globals():
return {
'now': datetime.utcnow(),
'today': date.today(),
'timedelta': timedelta,
'unread_notifs_count': unread_notifs_count,
'daycare_name': app.config.get('DAYCARE_NAME'),
'daycare_phone': app.config.get('DAYCARE_PHONE'),
'daycare_email': app.config.get('DAYCARE_EMAIL')
}
```

This makes daycare branding available in every template without needing to pass it explicitly in each route's `render_template` call.

8.4 Blueprint Architecture

Flask Blueprints structure the application into modular components. Each blueprint defines its own routes, templates, and before-request hooks. The project defines seven blueprints:

- `auth_bp` — handles `/login`, `/register`, `/logout`, `/reset-password`
- `admin_bp` — handles everything under `/admin/`
- `caregiver_bp` — handles everything under `/caregiver/`
- `parent_bp` — handles everything under `/parent/`
- `messages_bp` — handles `/messages` and `/api/messages/`
- `learning_bp` — handles everything under `/learning/`
- `api_bp` — handles everything under `/api/`

The `admin_bp` and `caregiver_bp` use the `before_request` hook with `@login_required` and `@role_required` applied at blueprint level, meaning every route in those blueprints is automatically protected without per-route decorator repetition. All blueprints are imported in `routes/__init__.py` and registered in `create_app`.

8.5 Security Implementation

Authentication: All sensitive routes are protected by `@login_required`. Unauthenticated requests are redirected to the login page.

Authorization: The `role_required(*roles)` decorator checks the current user's role against the allowed roles list and handles the caregiver/caretaker role alias. If the check fails, `abort(403)` is called, rendering the custom 403 error template.

Child-Level Isolation: The `get_authorized_child(child_id)` function enforces that caregivers can only access children assigned to them and parents can only access their own children. This is a data-level check, not only a route-level check. Both checks run independently — a user must pass the role check and the child ownership check.

Password Security: Werkzeug's `generate_password_hash` uses PBKDF2-SHA256 by default. This is computationally expensive to reverse, providing protection against offline dictionary attacks if the database were compromised.

Token Security: Password reset tokens use `secrets.token_urlsafe(32)`, providing 256 bits of entropy, suitable for security-sensitive one-time tokens.

File Upload Security: Uploaded files are validated by extension before saving. Files are saved with a UUID-prefixed name independent of the original filename, preventing path traversal. `werkzeug.utils.secure_filename` is applied as an additional sanitization step.

Error Handling: Custom handlers for 403, 404, and 500 prevent stack trace exposure. The 500 handler calls `db.session.rollback()` to ensure the database session is in a clean state.

8.6 File Upload Handling

The `save_uploaded_photo` utility in `routes/utils.py` implements the complete upload workflow:

```
def save_uploaded_photo(file):
if not file or file.filename == '':
return None
if not allowed_file(file.filename):
return None
ext = file.filename.rsplit('.', 1)[1].lower()
unique_filename = f"{uuid.uuid4().hex}_{secure_filename(file.filename)}"
upload_dir = current_app.config['PHOTO_UPLOAD_FOLDER']
os.makedirs(upload_dir, exist_ok=True)
filepath = os.path.join(upload_dir, unique_filename)
file.save(filepath)
return f"uploads/photos/{unique_filename}"
```

The `allowed_file` function checks that the extension is in the set `{png, jpg, jpeg, gif, webp}`. Flask's `MAX_CONTENT_LENGTH = 5 *1024* 1024` (5 MB) causes a 413 error if the content length header exceeds this value. The UUID-based filename prevents collisions even with identical original filenames.

8.7 QR Code Attendance System

Phase 1 — Generation:

When a child is enrolled, the `generate_qr_code` method creates a PNG file encoding `DAYCARE_CHECKIN:CHILD_ID={child.id}`, saved to `static/uploads/qrcodes/child_qr_{child.id}.png`. The relative path is stored in `child.qr_code_path` and displayed on the child's profile page as a scannable image.

The QR code image is generated with these parameters:

```
qr = qrcode.QRCode(version=1, box_size=10, border=2)
qr.add_data(f"DAYCARE_CHECKIN:CHILD_ID={self.id}")
qr.make(fit=True)
```

```
img = qr.make_image(fill_color="black", back_color="white")
img.save(filepath)
```

Phase 2 — Scanning:

The caregiver uses the QR scanner interface which activates the device camera through the browser's camera API and uses a JavaScript QR decoding library to read the code in real time. The decoded string is sent as a POST request to `/api/qr-scan`. The server parses the child ID, determines check-in or check-out state, updates the database, and returns a JSON confirmation displayed on screen.

8.8 Real-Time Notification System

Server Side — Emitting:

When an event occurs, the relevant route calls a function from `notifications.py`. The function creates a `Notification` record in the database and emits a WebSocket event to the user's personal room:

```
socketio_instance.emit('new_notification', notif.to_dict(), room=f"user_{user_id}")
```

Client Side — Receiving:

When a user's page loads, `main.js` connects to SocketIO and emits `join_user_notifications`. The server handler joins the client to `user_{user_id}`. When a `new_notification` event is received, JavaScript updates the badge count and prepends the notification to the dropdown without a page refresh.

Real-Time Chat:

When a user opens a chat thread, `chat.js` emits `join_child_chat` with the `child_id`. All users viewing that thread are in room `child_{child_id}`. When a user sends a message via the SocketIO `send_chat_message` event, it is broadcast to the room and rendered immediately in all connected windows. The `send_chat_message` handler in `app.py` validates the sender, determines the receiver, saves the message, broadcasts it to the room, and creates a Notification record for the receiver.

Chapter 9: Testing

9.1 Testing Strategy

The project includes a comprehensive test suite in `test_daycare.py` using Python's built-in `unittest` framework. The strategy covers integration testing of the full request-response cycle.

Test Configuration

A separate `TestConfig` class overrides the database URI to use an in-memory SQLite database:

```
class TestConfig(Config):
TESTING = True
WTF_CSRF_ENABLED = False
SQLALCHEMY_DATABASE_URI = 'sqlite://'
```

This ensures each test run starts with a fresh database and no test data can contaminate development or production data.

Test Fixture Setup

The `setUp` method creates the application with `TestConfig`, initializes the test client, creates all database tables, and seeds a known set of test data:

- 1 admin user
- 1 approved caregiver (Emma Thompson) and 1 pending caregiver
- 2 parents (David Miller, Elena Rostova)
- 2 children (Leo Miller — parent1 and caregiver1; Sophia Rostova — parent2 and caregiver1)
- 1 learning resource (Itsy Bitsy Spider)

The `tearDown` method removes the database session and drops all tables, ensuring complete isolation between tests.

9.2 Test Case Descriptions

Test 1 — Landing Page and Login Flow

Verifies the public landing page returns HTTP 200 and contains expected headline content. Tests that invalid credentials produce the expected error message. Tests that admin login redirects to the admin dashboard (verifying "Director Control Center" text appears). Tests that parent login redirects to the parent dashboard showing the enrolled child's name. Logout is called between each login to ensure session isolation.

Test 2 — Caregiver Pending Approval Flow

Tests the complete workflow: pending caregiver logs in and sees the "Caretaker Account Pending Review" page rather than the dashboard. Admin approves the caregiver through the admin route and receives a success flash message. The previously pending caregiver can now log in and see the "Caretaker Classroom Portal" dashboard.

Test 3 — Activity Log Flow

Tests that a caregiver can POST an activity log for an assigned child with meal type, food items, amount eaten, and notes. Verifies the logged food items appear on the child detail page. Then tests that the parent can navigate to the timeline for their child and find both the food items and the notes text in the response body.

Test 4 — Access Control Isolation

Tests that Parent 1 (David Miller) cannot access Sophia Rostova's data, which belongs to Parent 2:

- `GET /parent/child/{sophia_id}/timeline` — expects HTTP 403
- `GET /parent/child/{sophia_id}/photos` — expects HTTP 403
- `GET /messages/{sophia_id}` — expects HTTP 403

This test verifies data isolation at the most critical level of the system, confirming that the `get_authorized_child` function correctly raises 403 for unauthorized access.

Test 5 — Emergency Alert and Parent Banner

Tests that when a caregiver dispatches a critical emergency alert for a child, the parent's dashboard shows the expected alert banner text. Verifies "URGENT HEALTH / SAFETY ADVISORY" and the uppercased alert title "HIGH FEVER DETECTED" appear in the parent dashboard response.

Test 6 — Daily Attendance Flow

Tests the complete attendance lifecycle. Caregiver views the roster page (200, Leo Miller present in list). Caregiver posts a check-in action for Leo (200, flash mentions "checked in at"). Asserts directly in the database that `leo.is_checked_in` is True and `check_in_time` is not None. Caregiver posts a check-out action (200, flash mentions "checked out at"). Asserts `leo.is_checked_out` is True and `check_out_time` is not None.

Test 7 — Learning Resources Flow

Tests that the library index returns 200 and lists the seeded resource title "Itsy Bitsy Spider". Fetches the resource detail page and verifies the full rhyme text "The itsy bitsy spider climbed up the waterspout" is present in the response.

Test 8 — Password Reset Flow

Tests the complete reset lifecycle. Parent posts a reset request with their email (200, page shows "Reset Token Generated"). Asserts the user record in the database has a non-null reset_token. Parent posts a new password with the token (200, flash contains "password has been reset successfully"). Parent logs in with the new password and successfully sees their child's name on the dashboard.

9.3 Test Results Summary

Test ID	Description	Status
test_01	Landing page and login flow	Pass
test_02	Pending caregiver approval flow	Pass
test_03	Activity log creation and parent timeline	Pass
test_04	Cross-parent data isolation (403 enforcement)	Pass
test_05	Emergency alert and parent dashboard banner	Pass

Test ID	Description	Status
test_06	Attendance check-in and check-out lifecycle	Pass
test_07	Learning resource listing and detail view	Pass
test_08	Password reset and new password login	Pass

All 8 test cases pass on the development configuration using in-memory SQLite. The tests collectively exercise authentication, authorization, data creation, data isolation, real-time notification trigger logic, attendance state management, file-independent feature testing, and session security.

Chapter 10: Deployment

10.1 Local Development Setup

To run the platform in a local development environment:

Step 1: Clone the repository and navigate to the project directory.

Step 2: Create and activate a Python virtual environment.

```
python -m venv venv
venv\Scripts\activate (Windows)
source venv/bin/activate (Linux / macOS)
```

Step 3: Install all dependencies.

```
pip install -r requirements.txt
```

Step 4: Copy `.env.example` to `.env`. The default values are suitable for local development and do not need modification for demonstration purposes.

Step 5: Seed the development database with demonstration data.

```
python seed.py
```

Step 6: Start the development server.

```
python app.py
```

The application starts on `http://0.0.0.0:5000`. Open `http://localhost:5000` in a browser to access the landing page.

Default Test Credentials

Role	Email	Password	Notes
Admin	admin@daycare.com	Admin123!	Director account
Caregiver 1	caregiver1@daycare.com	Caregiver123!	Emma Thompson — approved
Caregiver 2	caregiver2@daycare.com	Caregiver123!	Marcus Vance — approved
Caregiver (pending)	pending@daycare.com	Caregiver123!	Jessica Alba — pending
Parent 1	parent1@example.com	Parent123!	David Miller, father of Leo
Parent 2	parent2@example.com	Parent123!	Elena Rostova, mother of Sophia and Maya
Parent 3	parent3@example.com	Parent123!	James Wilson, father of Noah

---

10.2 Environment Configuration

Variable	Default Value	Description
SECRET_KEY	virtual-daycare-secret...	Flask session signing key. Use a long random string in production.
DATABASE_URL	sqlite:///daycare.db	Database connection URI
SOCKETIO_ASYNC_MODE	threading	Async mode (threading or eventlet)
DAYCARE_NAME	Little Learners Virtual Daycare	Displayed in navigation and footer
DAYCARE_PHONE	+1 (555) 234-5678	Displayed in footer
DAYCARE_EMAIL	info@littlelearnersdaycare.com	Displayed in footer
DAYCARE_ADDRESS	123 Sunshine Lane...	Displayed in footer
ADMIN_EMAIL	admin@daycare.com	Used by init_db.py for initial admin
ADMIN_PASSWORD	AdminSecure2026!	Initial admin password
ADMIN_NAME	Admin	Initial admin display name

10.3 Database Initialization

Two scripts are provided for database management.

`seed.py` drops all tables, recreates them, and populates the database with comprehensive demonstration data. This is used during development and demonstrations. It generates 7 users, 4 children with QR codes, approximately 120 attendance records across 30 weekdays, 27 activity logs, 6 placeholder photos, 4 chat messages (including 1 emergency alert), 4 notifications, 8 learning resources, and 2 complaints.

`init_db.py` initializes a clean production database. It drops all existing data, recreates the schema, creates only the director admin account using credentials from environment variables, seeds the standard learning

curriculum, and produces no sample user or child data. This is run before the first production deployment.

10.4 Production Deployment

Heroku / Render / Railway Deployment

The `Procfile` is pre-configured:

```
web: gunicorn --worker-class eventlet -w 1 wsgi:app
```

The `eventlet` worker class is required for Flask-SocketIO WebSocket support. A single worker is required because SocketIO state is not shared between separate worker processes without a Redis message queue backend.

Set these environment variables in the platform's configuration panel:

- `SECRET_KEY` — a long random string (at least 32 characters)
- `DATABASE_URL` — the PostgreSQL connection string provided by the platform
- `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `ADMIN_NAME` — production admin credentials

PostgreSQL Migration

Setting `DATABASE_URL=postgresql://user:password@host:5432/dbname` is the only change required to use PostgreSQL. No code changes are needed. Run `python init_db.py` after setting this variable to create the tables and the initial admin account.

Static File Considerations

In production on platforms with ephemeral storage, the `static/uploads/` directory will be reset on each deployment. For long-term production use, uploaded files should be stored in an object storage service (AWS S3, Google Cloud Storage) and the URL stored in the database rather than a local file path. This requires replacing the `save_uploaded_photo` utility and the static file serving logic.

Chapter 11: Conclusion and Future Work

11.1 Conclusion

The Virtual Daycare Platform was developed as a comprehensive response to the real operational challenges faced by daycare centres in managing communication, documentation, and transparency between staff and families. The system successfully demonstrates a production-ready web application architecture using the Python Flask framework with real-time WebSocket communication, serving as a practical model for digitizing childcare management workflows.

The platform achieves all seven objectives defined at the outset of the project. Parents can monitor their child's daily activities in real time, view chronological activity timelines, browse categorized photo galleries, and track detailed attendance records with monthly statistics. Caregivers have a structured and intuitive workflow for logging every meaningful interaction with each assigned child, managing attendance through

both manual and QR code-based interfaces, uploading photographs, and escalating health concerns through the tiered emergency alert system. The director has a comprehensive administrative console covering enrolment, staff approval, complaint management, and learning resource curation.

The architecture is clean and extensible. The Application Factory pattern, Blueprint-based route organization, and model-layer business logic make the codebase maintainable and testable without tangled dependencies. The eight-integration-test suite provides high confidence in the correctness of all major workflows, and particularly in the access control isolation that prevents cross-user data exposure — the most critical security concern in a system handling children's personal and health data.

The QR code attendance system, real-time WebSocket notification delivery, child-centric messaging architecture, and structured activity logging system collectively represent a feature set competitive with commercial daycare management platforms, while remaining fully customizable and deployable without ongoing subscription costs.

11.2 Limitations

The following limitations exist in the current version.

Email and SMS Delivery: The external notification hooks for SMS and email are simulated. Urgent notifications print to the server log and console. A production deployment requires Twilio and SendGrid API credentials and integration work in the `send_sms` and `send_email` functions in `notifications.py`.

Single-Worker SocketIO Constraint: Flask-SocketIO with the eventlet async mode requires a single Gunicorn worker. This limits horizontal scalability of real-time features. For high-traffic production deployment, a Redis-backed SocketIO message queue would enable multi-worker and multi-instance deployment.

Ephemeral File Storage: Uploaded photos and QR codes are stored on the local filesystem. Platforms with ephemeral storage reset the filesystem on each deployment, causing uploaded files to be lost.

No Email Verification: Email verification at registration is not implemented. Any syntactically valid email can be used to create an account. A production system should verify email ownership before activating an account.

Single Daycare Centre: The platform manages a single daycare centre. There is no multi-tenancy support for managing multiple centres under one installation.

11.3 Future Enhancements

Production Email and SMS Integration: Replace the simulation hooks in `notifications.py` with actual Twilio and SendGrid API calls. This is the highest-priority enhancement for a real-world deployment. The function signatures are already in place; only the body of `send_sms` and `send_email` needs to be replaced.

Mobile Application: Develop a React Native or Flutter mobile application using the existing REST API endpoints. The `/api/` blueprint can be extended to support token-based authentication (JWT) for mobile clients, while the web interface continues to use session-based authentication.

Parent Registration Approval: Add an optional administrator approval step for parent account creation, mirroring the caregiver approval workflow. This would allow the daycare to verify parent relationships before granting access to child records.

Billing and Invoicing Module: Add a billing module that generates monthly invoices based on attendance records and allows parents to view billing history. Integration with Stripe for online payment would complete this feature set.

Video Documentation: Allow caregivers to record and upload short video clips (30 seconds to 2 minutes) from the activity logging form, giving parents richer documentation of milestones and special moments.

Redis-Backed SocketIO: Integrate Redis as the SocketIO message queue backend using `flask-socketio` with `message_queue='redis://localhost:6379'`. This removes the single-worker constraint and enables horizontal scaling across multiple server instances.

Object Storage Integration: Replace the local filesystem photo storage with an S3-compatible object storage service. The `save_uploaded_photo` function would be updated to upload to S3 and return a URL, which would be stored in the database instead of a local path.

Multi-Language Support: Implement Flask-Babel internationalization to support multiple languages, making the platform accessible to non-English speaking families and staff.

Government Compliance Reports: Generate attendance and incident reports in formats required by childcare licensing agencies, helping daycare operators meet their documentation obligations without manual record compilation.

Caregiver Shift Handoff Notes: Add a structured handoff notes feature where the outgoing caregiver leaves shift notes for the incoming caregiver about each child's current state, ensuring continuity of care across staff transitions.

Advanced Analytics Dashboard: Expand the admin dashboard with trend charts showing attendance rates over time, activity type distribution per child, most common health observations, and peak check-in and check-out times, supporting staffing and program planning decisions.

Parent Communication History Export: Allow parents to export a PDF of their child's activity logs for any date range, which could be shared with paediatricians or used for personal developmental records.

Staff Scheduling Module: Add a scheduling system allowing the administrator to define caregiver work schedules and automatically manage child-to-caregiver assignments when the primary caregiver is absent, preventing children from being unassigned during staff absences.

References

1. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.

2. Ronacher, A. (2024). Flask Documentation, Version 3.0. Pallets Projects. https://flask.palletsprojects.com/

3. SQLAlchemy Core and ORM Documentation (2024). Version 2.0. https://docs.sqlalchemy.org/

4. Flask-SQLAlchemy Documentation (2024). Version 3.1. https://flask-sqlalchemy.palletsprojects.com/

5. Flask-Login Documentation (2023). Version 0.6. https://flask-login.readthedocs.io/

6. Flask-SocketIO Documentation (2024). Version 5.x. https://flask-socketio.readthedocs.io/

7. Socket.IO (2024). Real-time bidirectional event-based communication documentation. https://socket.io/docs/

8. Twilio (2024). Twilio SMS Python Quickstart. https://www.twilio.com/docs/sms/quickstart/python

9. SendGrid (2024). Sending Email with SendGrid and Python. https://docs.sendgrid.com/for-developers/sending-email/api-getting-started

10. Bootstrap (2024). Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

11. Bootstrap Icons (2024). Open source SVG icon library. https://icons.getbootstrap.com/

12. python-qrcode (2024). GitHub Repository. https://github.com/lincolnloop/python-qrcode

13. Pillow Documentation (2024). PIL Fork — Python Imaging Library. https://pillow.readthedocs.io/

14. Python Software Foundation (2024). secrets — Secure random numbers for managing secrets. https://docs.python.org/3/library/secrets.html

15. Werkzeug Documentation (2024). Version 3.0. https://werkzeug.palletsprojects.com/

16. Gunicorn Documentation (2024). Python WSGI HTTP Server for UNIX. https://gunicorn.org/

17. OWASP (2023). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

18. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

19. Brightwheel (2024). Preschool Management Software. https://mybrightwheel.com/

20. Real Python (2023). Role-Based Access Control in Flask Applications. https://realpython.com/

Appendix

Appendix A: Database Seed Script Summary

Data Category	Count	Notes
Admin users	1	Director with full system access
Caregivers (approved)	2	Emma Thompson, Marcus Vance
Caregivers (pending)	1	Jessica Alba, for approval workflow demo
Parents	3	David Miller, Elena Rostova, James Wilson

Data Category	Count	Notes
Children	4	Leo (~2 yrs), Sophia (~3 yrs), Maya (~8 mo), Noah (~4 yrs)
Attendance records	~120	30 weekdays of history for all 4 children
Activity logs (today)	22	Across all 4 children
Activity logs (yesterday)	5	For date picker demonstration
Photo records	6	Generated placeholder images per child
Chat messages	4	Including 1 emergency alert
Notifications	4	Pre-seeded for parent and admin
Learning resources	8	Rhymes, stories, activities, game
Complaints	2	1 open (pending), 1 resolved with response

Appendix B: URL Route Reference

Blueprint	URL Pattern	Methods	Description
auth	/login	GET, POST	Login form
auth	/register	GET, POST	Registration form
auth	/logout	GET	Logout and session clear
auth	/pending-approval	GET	Pending caregiver waiting page
auth	/reset-password	GET, POST	Password reset request
auth	/reset-password/\<token\>	GET, POST	Password reset with token
admin	/admin/dashboard	GET	Admin metrics dashboard
admin	/admin/caregivers	GET	List all caregivers
admin	/admin/approve/\<id\>	GET, POST	Approve caregiver
admin	/admin/reject/\<id\>	GET, POST	Reject caregiver
admin	/admin/children	GET	List all children
admin	/admin/child/new	GET, POST	Enrol new child
admin	/admin/child/\<id\>/edit	GET, POST	Edit child profile
admin	/admin/complaints	GET	List complaints
admin	/admin/complaint/\<id\>/respond	POST	Respond to complaint
admin	/admin/learning	GET	List learning resources
admin	/admin/learning/new	GET, POST	Create learning resource

Blueprint	URL Pattern	Methods	Description
admin	/admin/learning/\<id\>/edit	GET, POST	Edit learning resource
admin	/admin/learning/\<id\>/delete	POST	Delete learning resource
caregiver	/caregiver/dashboard	GET	Caregiver home page
caregiver	/caregiver/child/\<id\>	GET	Child detail page
caregiver	/caregiver/child/\<id\>/log	GET, POST	Log activity
caregiver	/caregiver/child/\<id\>/photo	GET, POST	Upload photo
caregiver	/caregiver/child/\<id\>/attendance	GET, POST	Mark attendance
caregiver	/caregiver/attendance	GET	Daily roster
caregiver	/caregiver/child/\<id\>/alert	GET, POST	Emergency alert
caregiver	/caregiver/qr-scanner	GET	Redirect to roster
parent	/parent/dashboard	GET	Parent home page
parent	/parent/child/\<id\>/timeline	GET	Activity timeline
parent	/parent/child/\<id\>/photos	GET	Photo gallery
parent	/parent/child/\<id\>/attendance	GET	Attendance records
parent	/parent/child/\<id\>/profile	GET	Child profile and QR code
parent	/parent/complaints	GET, POST	Complaints and feedback
messages	/messages	GET	Thread list
messages	/messages/\<child_id\>	GET, POST	Chat thread
messages	/api/messages/\<child_id\>	GET	JSON message list
messages	/api/messages/\<child_id\>/send	POST	AJAX message send
learning	/learning/	GET	Learning library index
learning	/learning/\<id\>	GET	Resource detail
api	/api/notifications/unread	GET	JSON unread notifications
api	/api/notifications/mark-read/\<id\>	POST	Mark one notification read
api	/api/notifications/mark-all-read	POST	Mark all notifications read
api	/api/qr-scan	POST	Process QR attendance scan
root	/	GET	Landing page or role dashboard
root	/presentation	GET	Presentation page

Appendix C: Activity Type Reference

Type	Display	Icon Class	Use Case
meal	Meal	bi-cup-hot-fill	Breakfast, lunch, snack logging
nap	Nap / Rest	bi-moon-stars-fill	Rest periods with time and quality
mood	Mood	bi-emoji-smile-fill	Emotional state observations
diaper	Diaper / Bathroom	bi-droplet-fill	Diaper changes or potty success
note	Note	bi-journal-bookmark-fill	Health or behavioural observations
milestone	Milestone	bi-star-fill	Developmental achievements

Appendix D: Notification Events Reference

Trigger Event	Recipient	Urgent (SMS/Email)
Activity logged by caregiver	Child's parent	No
Photo uploaded	Child's parent	No
Child checked in (manual or QR)	Child's parent	No
Child checked out (manual or QR)	Child's parent	No
Emergency alert dispatched	Child's parent	Yes
New caregiver registered	All admins	No
Caregiver account approved	Approved caregiver	No
Child enrolled by admin	Child's parent	No
Parent complaint submitted	All admins	No
Admin responds to complaint	Submitting parent	No

Appendix E: Learning Resource Library — Default Seed Content

Title	Type	Age Group	Category
The Itsy Bitsy Spider	Rhyme	Infant	Music and Movement
Twinkle, Twinkle, Little Star	Rhyme	All Ages	Music and Movement
The Little Red Hen	Story	Toddler	Social and Emotional

Title	Type	Age Group	Category
The Three Little Pigs	Story	Preschool	Language and Imagination
Sensory Ice and Warm Water Exploration	Activity	Toddler	Sensory and Science
Rainbow Color Sorting Game	Activity	Preschool	Math and Logic
Tummy Time Mirror Exploration	Activity	Infant	Motor and Visual Skills
Simon Says: Animal Movement Edition	Game	Preschool	Gross Motor and Listening

---

End of Document

Document prepared for Little Learners Virtual Daycare — Virtual Daycare Platform

October 2026